

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

Normas

- ▶ Fechas de entrega y penalización:

Hasta el martes 3 de diciembre, 16:00 horas	0 %
Hasta el miércoles 4 de diciembre, 16:00 horas	20 %
Hasta el jueves 5 de diciembre, 16:00 horas	40 %

Después la puntuación máxima será 0
- ▶ Se comprobará plagio y se actuará sobre los detectados.
- ▶ Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- ▶ Todos los ejercicios de laboratorio se deben entregar a través de

`http://deliverit.fi.upm.es`

- ▶ El/los fichero(s) que hay que subir es/son `Paciente.java`, `Tests.java`, `UrgenciasAED.java`.
- ▶ La clase debe estar en el paquete `aed.urgencias` .
- ▶ La documentación de la API de `aedlib.jar` está disponible en

`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Tarea 1: Organizar un servicio de Urgencia

- ▶ Hay que implementar un sistema informático para manejar las colas de espera para recibir atención médica en un servicio de urgencias de un hospital.
- ▶ El interfaz Urgencias detalla los métodos a implementar.
- ▶ La primera tarea es *implementar* la interfaz Urgencias en una clase UrgenciasAED.java nueva que tenéis que crear. La clase debe estar en un fichero *nuevo*, UrgenciasAED.java.

Paciente

Un Paciente tiene los siguientes atributos, *getters* y/o *setters*:

- ▶ Un DNI
- ▶ Una prioridad que determina cómo de urgente es su caso, desde 0 (más urgente) a 10 (menos urgente)
- ▶ Dos “timestamps” —`tiempoAdmision` y `tiempoAdmisionEnPrioridad`— que guardan, respectivamente, la hora de llegada del paciente a urgencias, y el momento en el que se estableció su prioridad. Ambos son iguales en el momento de la admisión.

```
public class Paciente {  
    private String DNI;  
    private int prioridad;  
    private int tiempoAdmision;  
    private int tiempoAdmisionEnPrioridad;  
  
    public Paciente(String DNI, int prioridad,  
                    int tiempoAdmision,  
                    int tiempoAdmisionEnPrioridad) { ... }  
}
```

Completar la clase Paciente

- ▶ Falta completar los siguientes métodos en la clase Paciente:

```
public int compareTo(Paciente paciente) { ... }  
public boolean equals(Object obj) { ... }  
public int hashCode() { ... }
```

- ▶ Usad **solo** el DNI para comparar pacientes con equals y para calcular su hashCode().
- ▶ Comparación de urgencia de pacientes en compareTo:
 - ▶ primero, la prioridad (numéricamente, menor es más urgente),
 - ▶ en caso de igualdad, el tiempo en el nivel prioridad en que están (más tiempo de espera $\implies$ más urgente),
 - ▶ en caso de igualdad, el tiempo de admisión en las urgencias.
- ▶ Si atender a un paciente p1 es más urgente que atender a un paciente p2, p1.compareTo(p2) debe devolver un valor negativo (y positivo en el caso inverso).

Implementación de la clase Urgencias

- ▶ **Para esta tarea la eficiencia se considera fundamental.**
- ▶ En Urgencias.java se documentan los métodos a implementar y la **complejidad teórica máxima permitida en el peor caso (upper-bound)**.
- ▶ Asumimos que las operaciones put y get en tablas *hash* tienen complejidad $O(1)$.
- ▶ Varios métodos de las urgencias tienen un parámetro `int` hora que representa la hora en la que se realiza la llamada al método.

La interfaz Urgencias

- Operaciones $O(\log n)$:

```
public interface Urgencias {  
    // Admitir un nuevo paciente en urgencias  
    public Paciente admitirPaciente(String DNI, int prioridad, int hora)  
        throws PacienteExisteException;  
  
    // Paciente sale de urgencias sin ser atendido. Se  
    // borra de la estructuras de datos de las urgencias.  
    public Paciente salirPaciente(String DNI, int hora)  
        throws PacienteNoExisteException;  
  
    // Se atiende al primer paciente en la cola. El paciente se borra  
    // de las estructuras de datos de urgencias.  
    public Paciente atenderPaciente(int hora);  
  
    // Se cambia la prioridad de un paciente.  
    public Paciente cambiarPrioridad(String DNI, int nuevaPrio, int hora)  
        throws PacienteNoExisteException;  
}
```


La interfaz Urgencias

► Operaciones $O(1)$:

```
public interface Urgencias {  
  
    // Devuelve un paciente identificado mediante el DNI.  
    public Paciente getPaciente(String DNI);  
  
    // Devuelve un par con (i) la suma de los tiempos de espera desde  
    // la admision hasta ser atendido, para todos los pacientes que  
    // han sido atendidos, y (ii) el numero de pacientes atendidos.  
    public Pair<Integer,Integer> informacionEspera();  
}
```

La interfaz Urgencias

- Operaciones $O(n \log n)$:

```
public interface Urgencias {  
  
    // Aumenta la prioridad de los pacientes que han esperado mas que  
    // maxTiempoEspera en su prioridad actual.  
    public void aumentaPrioridad(int maxTiempoEspera, int hora);  
  
    // Devuelve un objeto Iterable ordenado segun el orden en la cola  
    public Iterable<Paciente> pacientesEsperando();  
}
```

Ejemplo

```
Urgencias u = new UrgenciasAED();

u.admitirPaciente("61969645T",6,1);
u.admitirPaciente("82772887P",6,10);
u.admitirPaciente("74939234Y",1,20);

u.atenderPaciente(30); ==> "74939234Y" (prioridad mas alta)
u.atenderPaciente(40); ==> "61969645T" (llegada mas pronto)

u.admitirPaciente("31825348F",9,50);
u.salirPaciente("82772887P",55);
u.atenderPaciente(60); ==> "31825348F" (anterior salio)

u.admitirPaciente("61569231M",9,61);
u.admitirPaciente("91862887R",2,62)
u.cambiarPrioridad("61569231M",0,63);

u.atenderPaciente(70); ==> "61569231M" (prioridad mas alta)
```

Ejemplo

```
Urgencias u = new UrgenciasAED();

u.admitirPaciente("61969645T",6,1);
u.admitirPaciente("82772887P",6,5);
u.admitirPaciente("74939234Y",2,10);

u.cambiarPrioridad("82772887P",1,20);

u.pacientesEsperando(); ==> ["82772887P","74939234Y","61969645T"]

u.atenderPaciente(30); ==> "82772887P"
u.atenderPaciente(40); ==> "74939234Y"

u.informacionEspera(); ==> Pair(55,2)

// Aumenta la prioridad para las pacientes que han esperado mas
// de 10 unidades de tiempo; la hora actual es 100.
u.aumentaPrioridad(10,100);
```

Tarea 2: Aprender a usar JUnit para probar APIs

- ▶ La segunda tarea de hoy es aprender a usar la librería JUnit 5 <https://junit.org/junit5/> para comprobar (test) el comportamiento de APIs.
- ▶ Concretamente, hay que añadir algunas pruebas (tests) al fichero `Tests.java` para comprobar determinadas funcionalidades del API de Urgencia.

Propiedades a comprobar #1

1. Comprueba que tras haber admitido a un paciente P_1 y después a un paciente P_2 , ambos con la misma prioridad, una llamada a `atenderPaciente()` devuelve el paciente P_1
2. Comprueba que tras haber admitido a un paciente P_1 y después a otro P_2 , ambos con la misma prioridad, una llamada al método `atenderPaciente()` devuelve el paciente P_1 primero y una segunda llamada devuelve el paciente P_2
3. Comprueba que después de haber admitido a un paciente P_1 con prioridad 5, y después a un paciente P_2 con prioridad 1, una llamada a `atenderPaciente()` devuelve el paciente P_2

Propiedades a comprobar #2

4. Comprueba que después de admitir a un paciente P_1 y otro P_2 , ambos con la misma prioridad, tras una llamada a `salirPaciente()` con el DNI del paciente P_1 como argumento, llamar al método `atenderPaciente()` devuelve el paciente P_2
5. Comprueba que tras admitir a un paciente P_1 y después a un paciente P_2 , ambos con prioridad 5, y después haber llamado al método `cambiarPrioridad()` con el DNI de P_2 y la nueva prioridad a 1, una llamada al método `atenderPaciente()` devuelve el paciente P_2

Escribiendo pruebas para Junit5

- ▶ Cada prueba se implementa en un método public sin argumentos y que no devuelve nada, es decir, de tipo void.
- ▶ Justo antes de la cabecera del método de prueba se pone una línea con la anotación “@Test”.
- ▶ Para comprobar que un valor devuelto por un método es correcto (y para notificar de un error si no lo es), se puede llamar al método

```
assertEquals(Object expected, Object actual)
```

donde `expected` es el valor correcto, y `actual` es el valor observado (devuelto)

- ▶ Es necesario importar los paquetes:

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.assertEquals;
```

- ▶ Hay muchas más “assertions”; si alguien tiene curiosidad:
<https://junit.org/junit5/docs/5.0.1/api/org/junit/jupiter/api/Assertions.html>.

Ejemplo de una Prueba

“Si admitimos solo a un paciente, una llamada al método atenderPaciente() devuelve el mismo paciente”:

```
package aed.urgencias;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class Tests {
    @Test
    public void testAdmitir() throws PacienteExisteException
    {
        Urgencias u = new UrgenciasAED();
        u.admitirPaciente("111", 5, 1);
        Paciente p = u.atenderPaciente(10);

        // Check expected DNI ("111") equals observed DNI (p.getDNI())
        assertEquals("111", p.getDNI());
    }
}
```

Notas Generales

- ▶ El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- ▶ Debe pasar todos los test TesterLab6 correctamente sin mensajes de error
- ▶ **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- ▶ Todos los ejercicios se corrigen manualmente antes de dar la nota final

¡Seguir estos consejos os permitirá conseguir mejores resultados!

- ▶ Corrección
- ▶ Ausencia de código repetido con la misma funcionalidad (podéis usar métodos auxiliares para evitarlo)
- ▶ Concisión y claridad del código
- ▶ Legibilidad, incluida selección de nombres descriptivos para variables y métodos
- ▶ El código debe estar correctamente indentado y con comentarios *útiles* cuando lo veáis necesario
- ▶ Eficiencia:
 - ▶ Se valorará la complejidad computacional del código
 - ▶ Se valorará no iterar innecesariamente en los recorridos de las estructuras de datos